CS C280 Computer Vision
Spring 2026 Alyosha Efros, Angjoo Kanazawa

This homework is due on Tuesday, Feb 24, 2025, at 11:59PM.

1 Introduction to Facial Keypoint Detection

Facial keypoint detection is a fundamental computer vision task that involves identifying the locations of important facial landmarks such as eyes, nose, and mouth corners in images. This task has numerous applications including face recognition, emotion analysis, augmented reality, animation, medical diagnosis, and more.

In this assignment, you will implement and compare different approaches for facial keypoint detection, ranging from direct coordinate regression to heatmap- based detection methods. You will also explore how transfer learning from pretrained models can improve performance on this task.

You will be working with a dataset of facial images, where each face has been annotated with 68 keypoints represented as (x, y) coordinates.

Resources including the starting skeleton *code* and the *dataset* are available **here**. **Note** that the provided skeleton code is for reference only. You may modify it as needed or implement your own solution from scratch. In all cases, your submission should be well-documented, reproducible, and clearly organized so that it is easy to follow.

2 Part 1: Direct Coordinate Regression (30 points)

In this part, you will implement a simple CNN architecture to directly regress the coordinates of facial keypoints. The model will take a grayscale image as input and output a vector of length $2K$, where K is the number of keypoints (68 in our case). This approach treats keypoint detection as a direct regression problem.

2.1 Implement a Simple CNN (15 points)

Design and implement a CNN with the following components:

- Convolutional layers with appropriate kernel sizes, strides, and padding
- Batch normalization for stable training
- Activation functions (ReLU, ELU, etc.)
- Pooling layers to reduce spatial dimensions
- Dropout for regularization
- Fully connected layers that output 136 values (68 keypoints $\times$ 2 coordinates)

2.2 Training and Evaluation (15 points)

- Implement a training loop that uses an appropriate loss function for regression (e.g., SmoothL1Loss or MSELoss)
- Train your model using the training dataset and validate on the test set
- Visualize the predicted keypoints alongside the ground truth on test images
- Analyze how different hyperparameters affect model performance
- Compute and report quantitative metric: Mean Squared Error (MSE)

3 Part 2: Transfer Learning for Keypoint Detection (40 points)

3.1 Pretrained ResNet Backbone (25 points)

Using transfer learning can significantly improve the performance of keypoint detection models. In this part, you will:

- Incorporate a pretrained ResNet model (e.g., ResNet-18 or ResNet-34) as the backbone for your keypoint detector
- Modify the input layer to handle grayscale images (or convert them to RGB)
- Remove the classification head and replace it with regression layers for keypoint prediction
- Implement a training strategy where you initially freeze the pretrained backbone and only train the new layers, then fine-tune the entire model
- Compare the performance with your Part 1 model in terms of accuracy and training efficiency

3.2 Advanced Pretrained Models (15 points)

For bonus points, implement one of the following advanced pretrained models as your backbone:

- DINO (self-supervised vision transformer)
- MAE (masked autoencoder)

Analyze how the self-supervised pretraining of these models affects keypoint detection performance compared to the supervised pretraining of ResNet models.

4 Part 3: Heatmap-based Keypoint Detection (30 points)

Direct coordinate regression has limitations in capturing the spatial structure of keypoints. In this part, you will implement a more sophisticated approach inspired by Mask R-CNN, where each keypoint is represented by a heatmap.

4.1 Heatmap Generation (10 points)

For each keypoint, create a heatmap by:

- Placing a Gaussian centered at the keypoint location
- Generating heatmaps of lower resolution than the input image (e.g., 64×64)
- Ensuring proper normalization of the heatmaps

4.2 U-Net Heatmap Prediction (20 points)

Implement a U-Net architecture that takes an image as input and outputs K heatmaps, one for each keypoint:

- Design an encoder-decoder architecture with skip connections
- Predict heatmaps for all keypoints simultaneously
- Extract keypoint coordinates from the predicted heatmaps using argmax
- Train the model using an appropriate loss function (e.g., MSE or binary cross-entropy)
- Visualize the predicted heatmaps alongside the ground truth
- Compare the performance with the direct regression approaches from Parts 1 and 2

5 Submission Guidelines

Your submission should include:

- Clearly categorized code for all three parts of the assignment (direct regression, transfer learning, heatmap-based)
- A report (PDF) that includes:
  - Description of your models and implementation details
  - Training and validation loss curves for each approach
  - Visualization of predictions on test images
  - Analysis of strengths and weaknesses of each method
  - Discussion of challenges faced and how you addressed them
  - Relevant code snippets included in-text where appropriate, and/or the full code attached as an appendix
- Model checkpoints for your best-performing models and/or clear documentation to reproduce results

6 Evaluation Criteria

Your assignment will be evaluated based on:

- Correctness of implementation (50%)
- Performance of your models (20%)
- Quality of analysis and visualizations (20%)
- Code organization and documentation (10%)